

flashjet

GPU-native jet clustering with the full merge history

Chirayu Gupta

ML4Jets 2026 · Vienna

September 2026

Benchmarks: **ATLAS Open Data** (CC0). Closures: analytic predictions on toy showers.

Summary — what this talk shows

A GPU implementation of the generalised- k_t algorithms (anti- k_t , k_t , Cambridge–Aachen) that clusters batches directly in device memory and keeps the **complete merge history** — so substructure follows as cheap array reads, not a second clustering pass.

Validation ladder, each rung closed against an independent reference:

rung	reference	headline
unit tests	independent NumPy tree-walks	129 passed, incl. GPU paths
analytic closures	leading-log predictions, toy shower	z_g on the $1/z$ curve; areas; β -ordering
vs FastJet	same events, same job	150/150 points, 100 % n_{jets} agreement
vs the experiment	ATLAS's own reconstructed jets	100 % match, ~ 600 clusters/event
speed	vectorised FastJet, identical input	39–99$\times$, growing with multiplicity

flashjet reproduces FastJet — and the experiment's own jets — while running roughly two orders of magnitude faster than per-jet CPU clustering. It is a **prototype**: validated, but not yet inside any experiment's software.

What flashjet is

Clustering is the CPU step in a GPU pipeline

Jet clustering sits on the critical path of nearly every jet-level ML pipeline — and is almost always done **off the accelerator**.

Four-momenta are copied to the host, clustered, copied back.

That round trip dominates when

- training revisits the same events for **many epochs**,
- jets are **reclustered** under many variations,
- clustering must sit **inside** a differentiable pipeline.

flashjet

Implements anti- k_t , k_t and C/A **natively on the GPU**.

Data never leaves device memory during clustering.

Triton/PyTorch. One entry point; runs unchanged on CPU and GPU.

It returns the tree, not just the jets

Clusters a **batch of jets or events at once**, and returns **two** things:

- 1 the particle $\rightarrow$ jet assignment;
- 2 the **complete merge history** — which pair merged, into what, at what distance.

Why the history matters

Groomed mass, z_g , R_g , Lund coordinates, exclusive subjects become **reads of that tree**.

No second clustering pass.

<code>jet_idx, n_jets</code>	particle-to-jet assignment
<code>hist_p1, hist_p2, hist_child</code>	which pair merged into what
<code>hist_d</code>	the d_{ij} at each merge step

One entry point

```
out = flashjet.cluster(p4, mask, R=1.0, algorithm="antikt")
#   p4           (B, N, 4) torch tensor | mask (B, N) bool
#   algorithm "antikt" | "kt" | "cambridge"

out.n_jets                # jets found
out.jets_p4(p4)           # jet four-momenta
out.exclusive_jets(n_jets=3) # F1
out.groomed_jets(p4, R, z_cut=0.1, beta=0.0) # F2 soft drop / mMDT
out.lund_coordinates(p4, R) # F3 primary Lund plane
```

Same call on CPU and GPU. jets_p4 is a scatter_add of constituents, so **gradients flow through the clustering**.

What is implemented

Clustering

- generalised- k_t : anti- k_t , k_t , C/A
- batched, ragged input via a mask
- three backends, auto-dispatched by size
- merge history recorded in-kernel
- jet regime *and* event regime

Substructure — reads of the history

- **F1** exclusive k_t subjets
- **F2** soft drop / mMDT / mass-drop
- **F3** primary Lund coordinates

All three are pure-torch post-reads: no kernel changes, CPU/GPU identical, negligible cost.

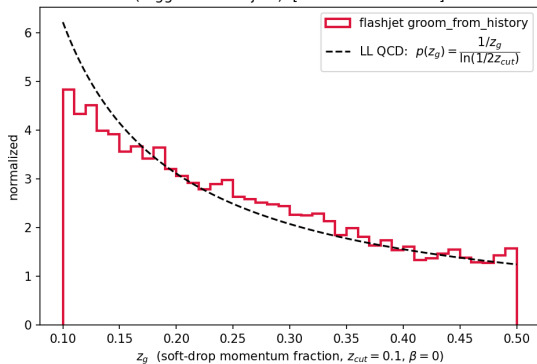
Status

A **prototype** — the physics is validated and the speedups are real, but it is a Python/Triton library, **not** integrated into any experiment's reconstruction framework.

Correctness

The history is right: closures against analytic predictions

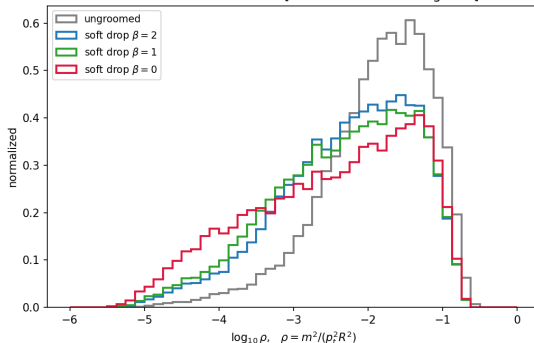
groomed splitting fraction vs the analytic prediction
(tagged 72% of jets) [cf. arXiv:1402.2657]



Soft-drop z_g follows the $1/z$ form predicted at leading-log, recovered from the C/A history.

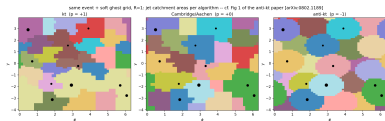
Toy-shower input, so the *prediction* is unambiguous — these test the decoders, not an event generator.

groomed jet mass: stronger grooming (smaller β) pushes the soft-radiation mass down [cf. arXiv:1402.2657 Figs 3-4]

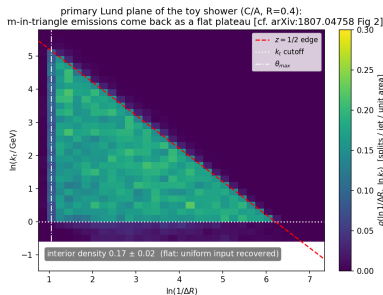


Groomed-mass ordering in β : larger β grooms less, so the mass peak moves up.

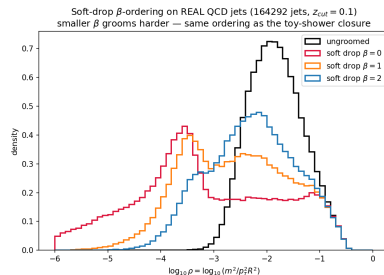
Clustering geometry



anti- k_t jet areas $\rightarrow \pi R^2$.



Lund plane uniform inside the kinematic triangle.



β -family ordering.

Independent NumPy tree-walks pin every decoder; the d -channel of the Lund output ties exactly to the splitting scales.

Same jets as FastJet — across the whole grid

3 algorithms $\times$ **5 radii** $\times$ **5 multiplicity bins** $\times$ **2 samples**, 20 000 jets per bin, on ATLAS Open Data.

check	result
number of jets found	100.000 % at every one of 150 points
leading-jet p_T within 10^{-4}	1.0000 at 148/150 points
median relative difference	$\sim 7 \times 10^{-8}$

k_t and C/A had never been checked against FastJet on real detector data — only against NumPy tree-walks. They now agree across the full radius $\times$ multiplicity grid.

The two exceptions are the *same single jet*: one constituent assigned across an R boundary. Jet counts still match and the leading jet is never ambiguous.

Same jets as the experiment

A stronger test than agreeing with FastJet.

ATLAS publishes an open dataset carrying both the **calorimeter clusters** and **its own reconstructed jets**. Cluster the ~ 600 clusters per event ourselves, and compare:

n_{jets} vs ATLAS	100.0 % match
mean jets/event	10.96 vs 10.96

This closes the algorithm against **the experiment's own published output** — not against another implementation of the same algorithm.

599.3 clusters/event: the event regime, where the problem is $O(N^2)$.

Speed

The benchmark

Data — ATLAS Open Data, freely presentable

sample	jets	$\langle n_{\text{const}} \rangle$
boosted top	30 000	59.2
QCD (light q/g)	30 000	52.4

Large- R jets with calorimeter-cluster constituents.

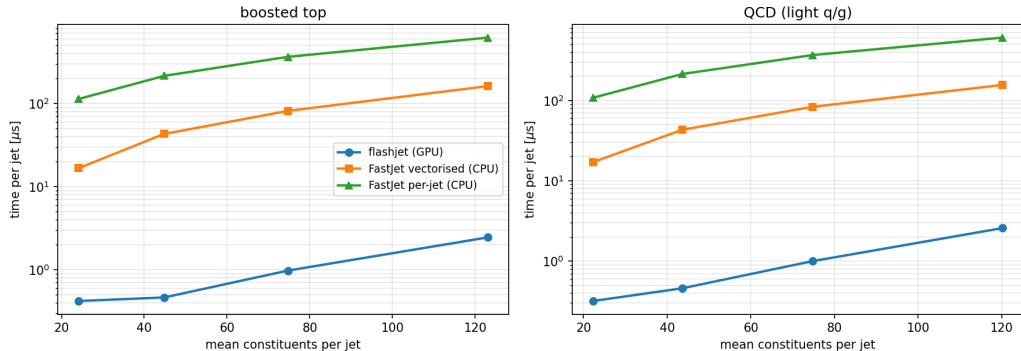
Method

- 1 extract constituents once, save;
- 2 cluster on GPU, time it, save the *same* events;
- 3 cluster those saved events with FastJet;
- 4 **check they agree** — else the timing is meaningless.

Both clusterers read the same saved file. Speedups are quoted against the **vectorised** FastJet interface — the faster, fairer baseline.

Time per jet

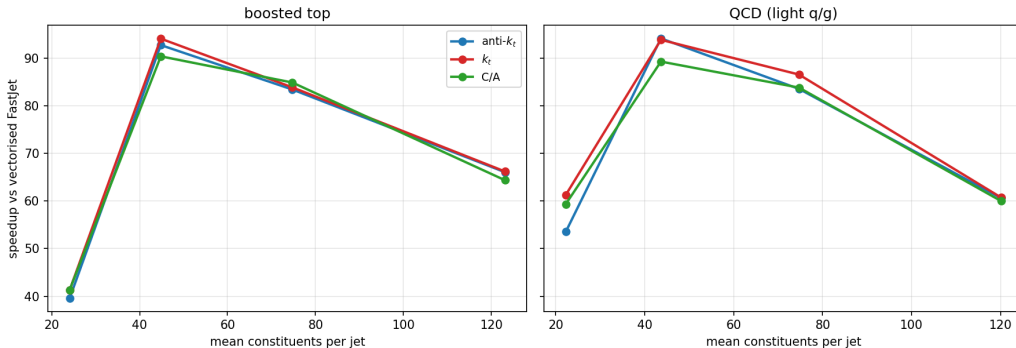
ATLAS Top Tagging Open Data — anti- k_t $R = 1.0$, Tesla V100S



Log scale. flashjet stays nearly flat while both FastJet interfaces climb steeply with multiplicity.

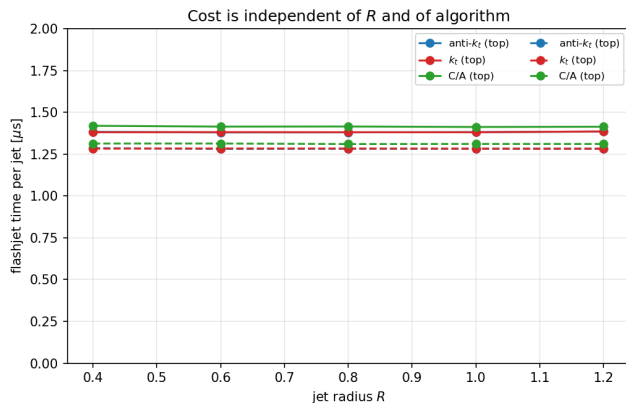
The advantage grows with jet complexity

Speedup vs multiplicity — $R = 1.0$, Tesla V100S (all points 100% n_{jets} agreement)



All three algorithms, both samples. Every point has **100%** n_{jets} agreement. Busier jets $\rightarrow$ bigger win — the regime that matters for boosted-object physics.

Cost is independent of R — and of the algorithm



Across $R = 0.4 \rightarrow 1.2$: a **0.3 % spread** over a $3\times$ range in R .

R changes *which* pairs merge, not *how many* distances are computed.

Across algorithms: anti- k_t , k_t and C/A agree to within $\sim 4\%$ at every multiplicity.

k_t and C/A timing is **free**.

The numbers

sample	alg	$\langle n \rangle$	flashjet [μ s/jet]	FastJet vec. [μ s/jet]	speedup
top	anti- k_t	24.1	0.427	16.5	39×
top	anti- k_t	44.8	0.464	43.0	93 ×
top	C/A	74.7	1.000	84.8	85×
top	C/A	123.1	2.565	165.1	64×
qcd	anti- k_t	22.2	0.346	17.5	51×
qcd	anti- k_t	43.6	0.463	45.7	99 ×
qcd	anti- k_t	74.7	1.215	89.8	74×
qcd	anti- k_t	120.2	2.673	154.9	58×

39–99× over vectorised FastJet (median 64×), with 100 % jet-count agreement everywhere. Benchmarked on an **NVIDIA V100-class** GPU.

$R = 1.0$ rows shown. Against the per-jet FastJet interface the gap is roughly *two orders of magnitude*.

- **The CPU baseline is FastJet's Python binding.** The production tool is C++. Until that is measured, the honest caveat is “*some of this could be Python overhead*” — **the single most valuable missing test.**
- **Large- R jets.** Boosted fat jets, ~ 50 – 120 constituents — not small- R jets.
- **Batch regime.** The advantage comes from clustering *many* jets at once. A per-event framework may not exploit that — it needs measuring, not assuming.
- **Hardware.** A single GPU generation was benchmarked; these are not the best numbers the library can produce.

Conclusions

- 1 **Clusters on the GPU and keeps the full merge history**, so substructure — exclusive subjects, soft drop, Lund coordinates — comes free as array reads.
- 2 **Gives the same jets as FastJet**. 150/150 grid points at 100 % n_{jets} agreement, across anti- k_t/k_t /C-A, $R = 0.4\text{--}1.2$ and 24–123 constituents per jet. k_t and C/A validated on real data for the first time.
- 3 **Reproduces the experiment's own reconstructed jets** exactly, at ~ 600 clusters per event.
- 4 **39–99 $\times$ faster** than vectorised FastJet, and the advantage *grows* with jet complexity. Cost is independent of R and of algorithm.
- 5 Still a **prototype**, with meaningful headroom left.

What is next

item	why
1 C++ FastJet baseline	every number here uses the Python binding; the production tool is C++
2 Tuning	the library has not been optimised for current hardware
3 Pile-up, grooming timing	untested axes a realistic workload will hit
4 Use inside experiment software	the real production test

Bottom line: as an accelerator-resident substitute for CPU clustering inside a learning workflow, flashjet is **correct** and roughly **two orders of magnitude faster** than per-jet CPU clustering.

Thank you

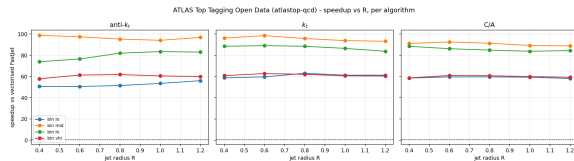
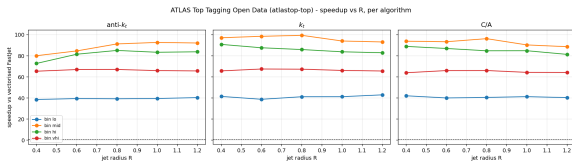
Questions?

Benchmarks: ATLAS Open Data (CC0).

Physics closures: analytic predictions on toy showers.

Backup

Backup — speedup vs R , per algorithm



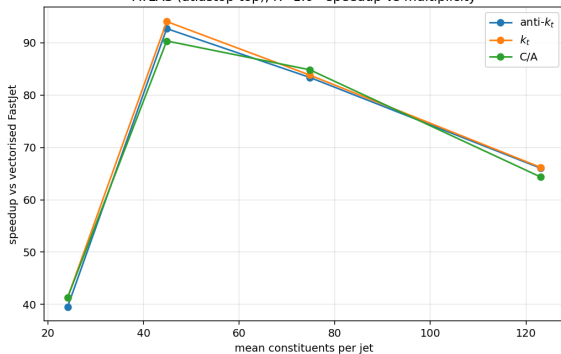
boosted top

Flat in R for all three algorithms; ordering is by multiplicity bin.

QCD (light q/g)

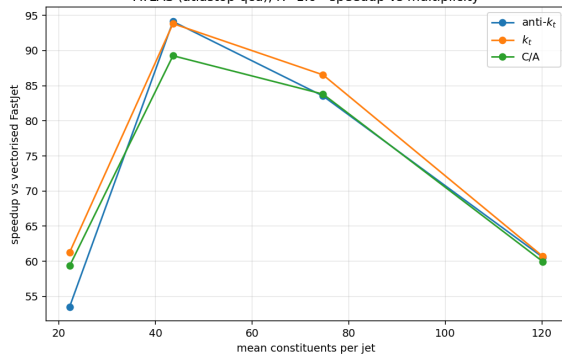
Backup — speedup vs multiplicity

ATLAS (atlastop-top), R=1.0 - speedup vs multiplicity



boosted top

ATLAS (atlastop-qcd), R=1.0 - speedup vs multiplicity



QCD (light q/g)

All benchmark data is **ATLAS Open Data** (CC0), read over the public redirector — no grid certificate required.

dataset	used for
ATLAS Top Tagging Open Data	timing + FastJet agreement
2020 ATLAS Jet Reconstruction	closure vs the experiment's own jets

Constituents are calorimeter clusters, massless. Physics closures use toy showers compared against leading-log analytic predictions, so the prediction being tested is unambiguous.